sLLM 파인튜닝 입문 가이드

비유로 배우는 지도 : 왜, 무엇을, 어떻게

초보자를 위한 개념 지도 + 실습 코드

전체 지도 한눈에 보기

이 가이드는 하나의 이야기를 따라갑니다 : 이미 다 자란 천재 학생(사전학습 모델)을 데려다가, 우리 회사에 필요한 인재로 다듬어가는 여정입니다. 아래 지도를 먼저 보고, 각 장이 왜 이 순서로 배치되었는지 확인한 뒤 본문으로 들어가겠습니다.

0장 · 왜 이 여정을 떠나야 하는가?

↓

1장 · 이 학생은 원래 어떤 사람인가 (구조)

↓

2장 · 어떻게 가르칠 것인가 (학습 도구·PEFT)

↓

3장 · 무엇을 가르칠 것인가 (학습 목적)

↓

4장 · 학습은 내부적으로 어떻게 일어나는가? (원리)

↓

5장 · 자원이 부족할 땐 어떻게 하는가 (최적화)

구분	핵심 질문	쓰인 비유	핵심 개념
Intro	왜 굳이 다듬어야 하나?	천재 신입사원	사전학습 vs 파인튜닝 비용
1장	이 학생은 어떤 사람인가?	통역팀 (청취/발화 담당)	encoder vs decoder, MLM vs CLM
2장	어떤 도구로 가르치나?	공장 우회 보정 장치	Full FT, LoRA·QLoRA·Adapter·Prompt
3장	무엇을 가르치나?	글쓰기 과외 · 기출문제집	SFT, 사전학습, RLHF·DPO

구분	핵심 질문	쓰인 비유	핵심 개념
4장	학습이 언제 일어나나?	검사 후 역추적	forward·backward·역전파
5장	자원 부족은 어떻게?	설계도 압축 창고	양자화, QLoRA

이 여정을 왜 떠나야 하는가?

사전학습 모델은 인터넷 전체를 읽고 자란, 상식과 언어능력이 뛰어난 신입사원과 같습니다. 하지만 이 신입사원을 그대로 투입할 수 없는 이유가 세 가지 있습니다.

① 범용 지능과 특수 지식 사이의 간극 : 아무리 똑똑해도 우리 회사의 출장 정산 규정, 재택근무 절차, 사내 시스템 이름은 태어나서 한 번도 들어본 적이 없습니다. Base LLM은 우리 회사의 고유 규정을 원래 알지 못합니다.

② 아는 것과 원하는 방식으로 말하는 것은 다른 문제 : 정보를 알고 있어도, 우리 회사가 원하는 형식(절차 순서, 어투, 근거 명시)으로 답하는 건 별개의 능력입니다. 이 간극을 메꾸는 것이 SFT이고, 더 나아가 여러 답변 중 어떤 게 더 우리 스타일에 맞는지 가려내는 것이 RLHF·DPO입니다.

③ 새로 키우는 것보다 데려다 다듬는 게 압도적으로 효율적 : 이 학생을 처음부터(백지 상태에서) 다시 키우는 것, 즉 사전학습부터 새로 하는 것은 막대한 데이터와 시간, 비용이 듭니다. 반면 이미 완성된 인재를 데려다 며칠짜리 사내 오리엔테이션(파인튜닝)만 시키는 건 비교할 수 없이 저렴하고 빠릅니다.

> **NOTE** 그래서 실무에서는 거의 항상 "이미 잘 자란 사전학습 모델 + 가벼운 파인튜닝" 조합을 씁니다. 1~5장은 바로 이 오리엔테이션을 어떻게 설계하고 실행하는지에 대한 이야기입니다.

1. 이 학생은 원래 어떤 사람인가? : 구조 이해

트랜스포머는 원래 번역을 위해 만들어졌고, 통역팀처럼 두 역할로 나뉘어 있었습니다.

청취 담당(인코더) : 원어민의 말을 처음부터 끝까지 다 듣고, 문장 전체의 맥락을 양방향으로 파악해 핵심 의미를 정리합니다. BERT가 바로 이 역할만 떼어낸 모델(encoder-only)입니다.

발화 담당(디코더) : 정리된 의미를 받아서, 다른 언어로 한 단어씩 순서대로 말을 만들어냅니다. GPT·Qwen이 이 역할만 떼어낸 모델(decoder-only)입니다.

BERT는 문장을 깊이 "이해"하는 데 특화되어 있어 분류·검색·개체명 인식에 쓰이고, Qwen 같은 decoder-only 모델은 답을 순차적으로 "생성"하는 데 특화되어 있어 챗봇·SFT·RLHF·DPO의 대상이 됩니다. 실습에서 다루는 모든 파인튜닝은 생성이 목적이므로 decoder-only(CLM) 모델을 사용합니다.

실습: 이 모델이 어떤 구조인지 코드로 확인하기

```
from transformers import AutoModelForCausalLM, AutoTokenizer

model = AutoModelForCausalLM.from_pretrained("internal-sllm-base")
tokenizer = AutoTokenizer.from_pretrained("internal-sllm-base")

print(model.config.model_type)          # 예: qwen2 (decoder-only, CLM)
print(model.config.is_encoder_decoder)  # False → 생성 전문가 구조
```

2. 어떻게 가르칠 것인가? : 학습 도구(PEFT)

공장의 핵심 기계(원본 가중치)를 통째로 뜯어고치는 대신, 그 옆에 작은 우회 보정 장치를 다는 방법이 있습니다. 이게 PEFT(Parameter-Efficient Fine-Tuning)입니다.

LoRA(Low-Rank Adaptation) **:** 핵심 기계 옆에 병렬로 다는 우회 보정 장치. 원본은 그대로 두고 작은 보정값만 학습하며, 나중에 원본과 합칠 수도 있어 속도 손해가 거의 없습니다.

Adapter : 각 작업 라인 중간에 직렬로 끼워 넣는 작은 보정 스테이션.

Prefix-Tuning : 모든 라인의 작업반장에게 참고 메모를 하나씩 쥐여주는 방식.

Prompt-Tuning : 공장 정문에 오늘의 작업 지시서만 붙여두는, 가장 가벼운 방식.

실무에서는 이 중 LoRA(및 QLoRA)가 사실상 표준으로 가장 널리 쓰입니다.

실습: LoRA 설정하기

```
from peft import LoraConfig, get_peft_model, TaskType

lora_config = LoraConfig(
    task_type=TaskType.CAUSAL_LM,
    r=8,                                    # 보정 장치의 손잡이 개수
    lora_alpha=16,                           # 보정값을 반영하는 증폭 다이얼
    lora_dropout=0.05,                       # 과적합 방지용 안전장치
    target_modules=["q_proj", "v_proj"],   # 보정 장치를 설치할 위치
)

model = get_peft_model(model, lora_config)
model.print_trainable_parameters()
# 예: trainable params: 4.2M || all params: 1.3B || trainable%: 0.32%
```

3. 무엇을 가르칠 것인가? : 학습 목적

도구(2장)를 정했다면, 이제 그 도구로 정확히 무엇을 가르칠지 구분해야 합니다.

SFT(Supervised Fine-Tuning, 지도 미세조정) **:** 모범답안집을 주고 따라 쓰게 하는 글쓰기 과외와 같습니다. (질문, 정답) 쌍으로 "이렇게 답하라"는 형식·태도를 가르칩니다.

CPT(Continued Pre-training, 지속 사전학습) **:** 라벨 없는 대량의 도메인 원문을 계속 읽혀서, 지식 자체를 두껍게 새겨 넣는 단계입니다. 새로운 방대한 사실 정보를 주입하려면 SFT보다 이 단계가 필요합니다.

RLHF / DPO(Reinforcement Learning from Human Feedback, 인간 피드백 기반 강화 학습 / Direct Preference Optimization 직접 선호 최적화) **:** 여러 답변 중 어떤 게 더 나은지 가려내는 단계입니다. RLHF는 채점 로봇(보상 모델)을 따로 만들어 실시간 코칭을 받는 방식이고, DPO는 채점 로봇 없이 "이 글이 저 글보다 낫다"는 비교 예시(기출문제집)를 직접 학습하는 더 단순한 방식입니다.

예시 : SFT 학습 실행하기

```
from trl import SFTTrainer

trainer = SFTTrainer(
    model=model,
    train_dataset=domain_dataset,   # (질문, 정답) 형태의 사내 데이터
    tokenizer=tokenizer,
)
trainer.train()
```

참고 : DPO 학습은 이렇게 생겼습니다

```
from trl import DPOTrainer

dpo_trainer = DPOTrainer(
    model=model,
    ref_model=None,                       # 비워두면 원본을 참조 모델로 자동 사용
    train_dataset=preference_dataset,   # (prompt, chosen, rejected) 비교 쌍
)
dpo_trainer.train()
```

실무 예시 : 지식 증류(Knowledge Distillation)를 SFT와 잇는 시나리오

지식 증류는 단독으로 끝나는 기법이 아니라, 부족한 도메인 데이터를 채워서 앞의 SFT 단계로 이어주는 전 단계입니다. 실제 현업에서는 보통 이런 순서로 연결됩니다.

① 데이터 부족 확인 : 사내 라벨 데이터가 수백 건 수준

↓

② 교사 모델에게 시드 예시 제공 : 소량의 실제 사례를 예시로 전달

↓

③ 유사 형식의 질문·답변 대량 생성 : 교사 모델이 합성 데이터 생성

↓

④ 품질 검수 : 사실 오류·형식 불일치·유해 표현 필터링

↓

⑤ 실제 데이터 + 합성 데이터를 합쳐 SFT·LoRA 학습

실습 : 교사 모델로 학습 데이터 만들고 SFT로 잇기

```
# ② + ③  교사 모델에게 소량의 실제 예시를 주고 합성 데이터 생성 요청
seed_examples = real_domain_dataset[:5]       # 실제 사내 데이터 일부를 예시로 사용

synthetic_data = []
for _ in range(200):
    prompt = build_generation_prompt(seed_examples)  # 예시 형식 프롬프트 구성
    response = teacher_model.generate(prompt)        # 대형 교사 모델 호출
    synthetic_data.append(parse_qa_pair(response))    # (질문, 답변) 쌍으로 파싱

# ④  품질 검수 (형식·사실관계 등 필터링)
clean_data = [d for d in synthetic_data if quality_check(d)]

# ⑤  실제 데이터 + 합성 데이터를 합쳐 SFT 학습 (앞서 본 흐름 그대로 재사용)
domain_dataset = real_domain_dataset + clean_data
trainer = SFTTrainer(
    model=model,
    train_dataset=domain_dataset,
    tokenizer=tokenizer,
)
trainer.train()
```

교사 모델(teacher model)은 특정 상용 LLM을 가리키는 고유명사가 아니라, "학생 모델보다 더 크고

뛰어난 모델"이라는 역할을 가리키는 이름입니다. 이 역할을 GPT-4나 Claude 같은 상용 API가 맡을 수도 있고, 회사가 이미 갖고 있는 더 큰 자체 학습 모델이 맡을 수도 있습니다. 즉 교사가 꼭 외부 상용 서비스일 필요는 없습니다.

가이드의 시나리오에서 상용 범용 LLM을 예로 든 이유는, 그게 지식 증류의 정의여서가 아니라 "사내에 마땅한 대형 모델이 따로 없는 경우가 많고, 상용 LLM은 이미 폭넓은 지식과 추론 능력을 갖추고 있어 데이터가 부족할 때 가장 빠르게 채울 수 있는 흔한 선택지"이기 때문입니다.

지식 증류의 구도는 "학생(우리가 학습시키려는 작은 sLLM)"과 "교사(더 크고 뛰어난 어떤 모델이든)"로 나뉩니다. 교사에게 몇 개의 실제 예시(시드)를 보여주고 "이런 형식으로 더 만들어줘"라고 요청하면, 교사가 자신의 지식과 추론 능력으로 그럴듯한 (질문, 답변) 쌍을 대량 생성해줍니다. 그 결과물을 학생 모델이 SFT로 학습하는 것이 코드의 teacher_model.generate(prompt) 부분이고, 여기서 teacher_model 자리에는 상용 API든 사내 대형 모델이든 상황에 맞는 걸 넣으면 됩니다.

비유로 정리하면, 교사 모델은 "이미 그 일을 잘 아는 어떤 선배든"을 가리키고, 실무에서는 우연히 그 선배 역할을 상용 LLM이 맡는 경우가 많을 뿐입니다. 학생 모델은 그 선배가 만들어준 예상문제·모범답안으로 공부하는 신입사원(우리가 다듬으려는 작은 모델)입니다.

> **NOTE** 교사 모델도 모르는 지식(우리 회사만의 고유 규정 등)은 증류로 주입할 수 없습니다. 또한 상용 API를 교사로 쓸 경우 이용 약관상 그 출력으로 경쟁 모델을 학습시키는 것이 제한될 수 있어 사전에 확인이 필요합니다.

4. 학습은 내부적으로 어떻게 일어나는가? : 원리

완성된 제품(모델 출력)을 검사해서 오차를 확인한 뒤, 그 오차가 왜 생겼는지 거꾸로 추적하며 보정 장치의 손잡이를 얼마나 돌려야 할지 계산하는 과정, 이것이 역전파(Backpropagation)입니다.

① Forward : 데이터가 모델을 통과해 예측 생성

↓

② Loss 계산 : 예측과 정답을 비교해 오차 산출

↓

③ Backward : 오차를 거슬러 올라가며 조정 방향 계산

↓

④ Optimizer Step : 보정 장치(LoRA)만 실제로 업데이트

참고로 이 순서(forward→backward)는 모든 신경망에 공통된 절차이며, RNN만의 특징이 아닙니다. RNN에 고유한 것은 시퀀스의 시간축까지 함께 거슬러 올라가는 BPTT(Backpropagation Through Time, 시간 역전파)라는 변형이고, Transformer는 이런 시간축 순환 구조가 없어 일반적인 역전파만으로 충분합니다.

예시 : 학습 루프 한 스텝

```
outputs = model(**batch)      # ① Forward
loss = outputs.loss           # ② 오차 계산
loss.backward()               # ③ Backpropagation
optimizer.step()              # ④ 파라미터(LoRA만) 업데이트
optimizer.zero_grad()         # 다음 스텝을 위해 기울기 초기화
```

5. 자원이 부족할 땐 어떻게 하는가? : 최적화

LoRA에서는 원본 핵심 기계(가중치)를 고해상도 설계도(16비트) 그대로 창고에 보관합니다. 모델이 커질수록 창고 공간(GPU 메모리)을 많이 차지합니다.

QLoRA(Quantized Low-Rank Adaptation, 양자화된 저순위 적응)는 이 설계도를 4비트짜리 초압축 요약본으로 창고에 보관하고, 계산이 필요한 순간에만 잠깐 펼쳐서 쓴 뒤 다시 압축해둡니다. 학습 대상(LoRA 보정 장치)은 완전히 동일하며, 원본을 얼마나 압축해서 보관하느냐만 다릅니다.

장점 : 메모리 사용량이 극적으로 줄어 훨씬 큰 모델을 훨씬 저사양 GPU에서도 파인튜닝할 수 있습니다.

트레이드오프 : 압축·해제 과정에서 계산 속도가 약간 느려지고, 아주 미세한 정보 손실이 생길 수 있습니다.

예시 : QLoRA 설정하기

```
from transformers import AutoModelForCausalLM, BitsAndBytesConfig

bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",
    bnb_4bit_compute_dtype="bfloat16",
)

model = AutoModelForCausalLM.from_pretrained(
    "internal-sllm-base",
    quantization_config=bnb_config,   # 창고를 4비트로 압축
)

model = get_peft_model(model, lora_config)  # 이후 LoRA는 2장과 동일하게 적용
```

맺음말

Intro에서 왜 이 여정이 필요한지 확인했고, 1장에서 학생의 구조를, 2장에서 가르치는 도구를, 3장에서 가르칠 목적을, 4장에서 학습의 원리를, 5장에서 자원 제약을 푸는 법을 차례로 살펴봤습니다.

각 장은 다음 장의 전제 조건이 되도록 설계되어 있으므로, 순서대로 따라가면 개별 개념에 매몰되지 않고 전체 숲을 놓치지 않을 수 있습니다. 실습 코드는 모두 최소 예시이며, 실제 프로젝트에서는 사내 데이터 형식과 자원 상황에 맞게 각 설정값을 조정해서 사용하시면 됩니다.